TaskFlow

Team Task Manager - a full-stack Java project, from local code to the cloud

Author: **Abel Haile Magicho**

Stack: **Java 17**, **Spring Boot 3**, **Maven**, **Spring Web**, **Spring Data JPA**, **Bean Validation**, the **H2 database** for local work and **PostgreSQL** for containers and the cloud.

This explainer walks the whole journey in order. It shows what the project is, how the code is put together, the real output of a real run on a real machine, and the exact steps that take the same artifact to **Docker**, to **Kubernetes**, to **Microsoft Azure** and to **Amazon Web Services**.

What the project is

TaskFlow is a team task manager. A user opens one web page, adds tasks, watches the counts change, marks work done and deletes what is finished. Behind the page sits a **REST API** with full **CRUD** behaviour: create, read, update and delete.

Every task carries seven facts: an **id**, a **title**, a **description**, a **status**, a **priority**, a **due date** and a **created-at timestamp**. The status is one of **TODO**, **IN_PROGRESS** or **DONE**. The priority is one of **LOW**, **MEDIUM** or **HIGH**.

The whole product ships as **one deployable artifact**. The frontend is a single static page that the Spring application serves itself, so the backend and the frontend travel together in one jar file and later in one container image.

One artifact is the idea that carries the whole project. Build it once, then run the identical artifact on a laptop, in Docker, in Kubernetes and in both clouds.

The API at a glance

The list endpoint lives at **/api/tasks** and accepts an optional status filter, so a request for the tasks with status DONE returns only finished work. The **/api/tasks/stats** endpoint returns the counts: total, to do, in progress and done. A create request answers with status **201**. A delete request answers with status **204**. A request for a missing task answers with status **404**. A create request with a blank title fails validation and answers with status **400**.

Architecture

The browser talks only to the Spring application. The page loads from the application itself, and the JavaScript on the page calls the REST API with JSON. There is no separate frontend server and no second deployment.

Inside the application the work splits into three layers. The **controller** layer receives HTTP requests and returns JSON. The **repository** layer talks to the database through Spring Data JPA. The **model** layer holds the Task entity, which is both the database shape and the JSON shape.

The database changes with the environment, and the code does not. The default local profile uses the **H2 in-memory database**, which needs zero setup and resets on every restart. The **postgres profile** points the same code at PostgreSQL, and that is the profile Docker Compose, Kubernetes, Azure and AWS all use.

Trap: H2 data vanishes on restart. It is a development database, never a production one.

The journey from local code to the cloud

Stage one is local code. Maven compiles the Java sources, runs the tests and packages one executable jar. The developer runs the jar, opens the page and works against H2.

Stage two is Docker. A multi-stage Dockerfile builds the jar inside a Maven image, then copies only the jar into a small Java runtime image. Docker Compose adds a PostgreSQL container next to the application container and wires the two together with environment variables and a health check.

Stage three is Kubernetes. The same image runs under a Deployment with two replicas. A Service gives the pods one stable address, an Ingress exposes the app on a host name, a ConfigMap carries the settings and a Secret carries the database password. A second Deployment runs PostgreSQL inside the cluster for demos.

Stage four is the cloud, twice. On Azure the image is built and stored in Azure Container Registry and deployed to Azure Kubernetes Service with the same manifests. On AWS the image is pushed to Elastic Container Registry and runs either on Elastic Kubernetes Service with the same manifests or as a serverless Fargate service on Elastic Container Service.

Stage five is automation. A GitHub Actions workflow builds and tests every push, builds the Docker image, and then runs the Azure and AWS deploy stages with credentials read from repository secrets.

Key idea: the manifests never change between AKS and EKS. Only the image address changes.

The main code files in full

The files below are the real files from the project, printed in full. Code keeps its own punctuation, exactly as it runs.

src/main/java/com/abel/taskflow/model/Task.java

```
package com.abel.taskflow.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.PrePersist;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

import java.time.LocalDate;
import java.time.LocalDateTime;

@Entity
@Table(name = "tasks")
public class Task {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Title is required")
    @Size(max = 120, message = "Title must be at most 120 characters")
    @Column(nullable = false)
    private String title;

    @Size(max = 1000, message = "Description must be at most 1000 characters")
    private String description;

    @NotNull(message = "Status is required")
    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private TaskStatus status = TaskStatus.TODO;

    @NotNull(message = "Priority is required")
    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Priority priority = Priority.MEDIUM;

    private LocalDate dueDate;

    @Column(nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @PrePersist
    void onCreate() {
        this.createdAt = LocalDateTime.now();
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }
```

```
    public void setTitle(String title) {
        this.title = title;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public TaskStatus getStatus() {
        return status;
    }

    public void setStatus(TaskStatus status) {
        this.status = status;
    }

    public Priority getPriority() {
        return priority;
    }

    public void setPriority(Priority priority) {
        this.priority = priority;
    }

    public LocalDate getDueDate() {
        return dueDate;
    }

    public void setDueDate(LocalDate dueDate) {
        this.dueDate = dueDate;
    }

    public LocalDateTime getCreatedAt() {
        return createdAt;
    }
}
```

The entity is the heart of the project. The **@Entity** annotation makes the class a database table. The validation annotations are the gate at the API door: a blank title or an oversized title never reaches the database.

Trap: the created-at field is set by the @PrePersist hook, never by the client. A client that sends a created-at value is ignored.

src/main/java/com/abel/taskflow/controller/TaskController.java

```
package com.abel.taskflow.controller;

import com.abel.taskflow.model.Task;
import com.abel.taskflow.model.TaskStatus;
import com.abel.taskflow.repository.TaskRepository;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PatchMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;

import java.util.LinkedHashMap;
```

```
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/tasks")
public class TaskController {

    private final TaskRepository repository;

    public TaskController(TaskRepository repository) {
        this.repository = repository;
    }

    @GetMapping
    public List<Task> list(@RequestParam(required = false) TaskStatus status) {
        if (status != null) {
            return repository.findByStatus(status);
        }
        return repository.findAll();
    }

    @GetMapping("/stats")
    public Map<String, Object> stats() {
        Map<String, Object> result = new LinkedHashMap<>();
        result.put("total", repository.count());
        result.put("todo", repository.countByStatus(TaskStatus.TODO));
        result.put("inProgress", repository.countByStatus(TaskStatus.IN_PROGRESS));
        result.put("done", repository.countByStatus(TaskStatus.DONE));
        return result;
    }

    @GetMapping("/{id}")
    public Task get(@PathVariable Long id) {
        return repository.findById(id)
                .orElseThrow(() -> new ResponseStatusException(
                        HttpStatus.NOT_FOUND, "Task not found: " + id));
    }

    @PostMapping
    public ResponseEntity<Task> create(@Valid @RequestBody Task task) {
        task.setId(null);
        Task saved = repository.save(task);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }

    @PutMapping("/{id}")
    public Task update(@PathVariable Long id, @Valid @RequestBody Task incoming) {
        Task existing = get(id);
        existing.setTitle(incoming.getTitle());
        existing.setDescription(incoming.getDescription());
        existing.setStatus(incoming.getStatus());
        existing.setPriority(incoming.getPriority());
        existing.setDueDate(incoming.getDueDate());
        return repository.save(existing);
    }

    @PatchMapping("/{id}/complete")
    public Task complete(@PathVariable Long id) {
        Task existing = get(id);
        existing.setStatus(TaskStatus.DONE);
        return repository.save(existing);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        if (!repository.existsById(id)) {
            throw new ResponseStatusException(
                    HttpStatus.NOT_FOUND, "Task not found: " + id);
        }
        repository.deleteById(id);
        return ResponseEntity.noContent().build();
    }
}
```

The controller is thin on purpose. It validates the request body, delegates storage to the repository and translates a missing task into a 404 answer. The complete endpoint exists so the page can mark a task done with one small request instead of a full replace.

src/main/resources/application.properties

```
spring.application.name=taskflow
server.port=8080

# Default local profile: in-memory H2 database, zero setup.
spring.datasource.url=jdbc:h2:mem:taskflow;DB_CLOSE_DELAY=-1
spring.datasource.username=sa
spring.datasource.password=
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
spring.h2.console.enabled=true
```

src/main/resources/application-postgres.properties

```
# Postgres profile: used by Docker Compose, Kubernetes, Azure and AWS.
# Activate with SPRING_PROFILES_ACTIVE=postgres
spring.datasource.url=${SPRING_DATASOURCE_URL:jdbc:postgresql://localhost:5432/taskflow}
spring.datasource.username=${SPRING_DATASOURCE_USERNAME:taskflow}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD:taskflow}
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
spring.h2.console.enabled=false
```

Two property files, two environments. The postgres file reads its address and credentials from environment variables, which is how containers and Kubernetes inject settings without touching code.

Dockerfile

```
# ---------- Stage 1: build with Maven on JDK 17 ----------
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -B -q dependency:go-offline
COPY src ./src
RUN mvn -B -q -DskipTests package

# ---------- Stage 2: small JRE runtime image ----------
FROM eclipse-temurin:17-jre
WORKDIR /app
RUN groupadd -r taskflow && useradd -r -g taskflow taskflow
COPY --from=build /app/target/taskflow-1.0.0.jar app.jar
USER taskflow
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

The Dockerfile has two stages. Stage one builds with Maven on a full JDK image. Stage two runs the jar on a small JRE image, as a non-root user. The final image stays small and carries no build tools.

Trap: the dependency step runs before the source copy on purpose. Docker then caches the downloaded dependencies and rebuilds stay fast.

docker-compose.yml

```
services:
  db:
    image: postgres:16
    container_name: taskflow-db
    environment:
      POSTGRES_DB: taskflow
      POSTGRES_USER: taskflow
      POSTGRES_PASSWORD: taskflow
    ports:
      - "5432:5432"
    volumes:
      - taskflow-db-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U taskflow -d taskflow"]
      interval: 5s
      timeout: 3s
      retries: 10

  app:
    build: .
    container_name: taskflow-app
    depends_on:
      db:
        condition: service_healthy
    environment:
      SPRING_PROFILES_ACTIVE: postgres
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/taskflow
      SPRING_DATASOURCE_USERNAME: taskflow
      SPRING_DATASOURCE_PASSWORD: taskflow
    ports:
      - "8080:8080"

volumes:
  taskflow-db-data:
```

Compose runs the real pairing: the application plus PostgreSQL. The application waits for the database health check, and the database keeps its data in a named volume across restarts.

k8s/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: taskflow
  labels:
    app: taskflow
spec:
  replicas: 2
  selector:
    matchLabels:
      app: taskflow
  template:
    metadata:
      labels:
        app: taskflow
    spec:
      containers:
        - name: taskflow
          # Replace with your registry image, for example:
          # myregistry.azurecr.io/taskflow:1.0.0  or
          # 123456789012.dkr.ecr.us-east-1.amazonaws.com/taskflow:1.0.0
          image: taskflow:1.0.0
          ports:
            - containerPort: 8080
          envFrom:
            - configMapRef:
                name: taskflow-config
            - secretRef:
                name: taskflow-secret
          readinessProbe:
            httpGet:
              path: /
              port: 8080
            initialDelaySeconds: 15
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /
              port: 8080
            initialDelaySeconds: 30
            periodSeconds: 20
          resources:
            requests:
              cpu: "250m"
              memory: "512Mi"
            limits:
              cpu: "1"
              memory: "1Gi"
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: taskflow-db
  labels:
    app: taskflow-db
spec:
  replicas: 1
  selector:
    matchLabels:
      app: taskflow-db
  template:
    metadata:
      labels:
        app: taskflow-db
    spec:
      containers:
        - name: postgres
          image: postgres:16
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_DB
```

```
            value: taskflow
          - name: POSTGRES_USER
            value: taskflow
          - name: POSTGRES_PASSWORD
            valueFrom:
              secretKeyRef:
                name: taskflow-secret
                key: POSTGRES_PASSWORD
        resources:
          requests:
            cpu: "250m"
            memory: "256Mi"
          limits:
            cpu: "500m"
            memory: "512Mi"
```

The Deployment file holds two workloads: the application with two replicas, health probes and resource limits, and PostgreSQL with one replica. Both read the password from the same Secret.

Trap: the image value in this file is a placeholder. Replace it with the real registry address before applying, or the pods will never start.

.github/workflows/ci-cd.yml

```
name: TaskFlow CI/CD

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:

env:
  IMAGE_NAME: taskflow
  IMAGE_TAG: ${{ github.sha }}

jobs:
  build-and-test:
    name: Build, test and package
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "17"
          cache: maven

      - name: Build and run tests
        run: mvn -B verify

      - name: Build Docker image
        run: docker build -t ${{ env.IMAGE_NAME }}:${{ env.IMAGE_TAG }} .

  deploy-azure-aks:
    name: Deploy to Azure AKS
    needs: build-and-test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: azure
    steps:
      - uses: actions/checkout@v4

      - name: Log in to Azure
        uses: azure/login@v2
        with:
          creds: ${{ secrets.AZURE_CREDENTIALS }}
```

```
    - name: Log in to Azure Container Registry
      run: az acr login --name ${{ secrets.ACR_NAME }}

    - name: Push image to ACR
      run: |
        IMAGE=${{ secrets.ACR_LOGIN_SERVER }}/${{ env.IMAGE_NAME }}:${{ env.IMAGE_TAG }}
        docker build -t $IMAGE .
        docker push $IMAGE

    - name: Connect kubectl to AKS
      run: |
        az aks get-credentials \
          --resource-group ${{ secrets.AKS_RESOURCE_GROUP }} \
          --name ${{ secrets.AKS_CLUSTER_NAME }} --overwrite-existing

    - name: Deploy the Kubernetes manifests
      run: |
        kubectl apply -f k8s/configmap.yaml
        kubectl apply -f k8s/secret.yaml
        kubectl apply -f k8s/deployment.yaml
        kubectl apply -f k8s/service.yaml
        kubectl apply -f k8s/ingress.yaml
        kubectl set image deployment/taskflow \
          taskflow=${{ secrets.ACR_LOGIN_SERVER }}/${{ env.IMAGE_NAME }}:${{ env.IMAGE_TAG }}
        kubectl rollout status deployment/taskflow

deploy-aws:
  name: Deploy to AWS EKS and ECS
  needs: build-and-test
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  environment: aws
  steps:
    - uses: actions/checkout@v4

    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
        aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
        aws-region: ${{ secrets.AWS_REGION }}

    - name: Log in to Amazon ECR
      id: ecr
      uses: aws-actions/amazon-ecr-login@v2

    - name: Push image to ECR
      run: |
        IMAGE=${{ steps.ecr.outputs.registry }}/${{ env.IMAGE_NAME }}:${{ env.IMAGE_TAG }}
        docker build -t $IMAGE .
        docker push $IMAGE
        echo "IMAGE=$IMAGE" >> $GITHUB_ENV

    # Option A: Kubernetes on EKS, reusing the same k8s manifests
    - name: Deploy to EKS
      run: |
        aws eks update-kubeconfig \
          --name ${{ secrets.EKS_CLUSTER_NAME }} --region ${{ secrets.AWS_REGION }}
        kubectl apply -f k8s/configmap.yaml
        kubectl apply -f k8s/secret.yaml
        kubectl apply -f k8s/deployment.yaml
        kubectl apply -f k8s/service.yaml
        kubectl apply -f k8s/ingress.yaml
        kubectl set image deployment/taskflow taskflow=$IMAGE
        kubectl rollout status deployment/taskflow

    # Option B: serverless containers on ECS Fargate
    - name: Deploy to ECS Fargate
      run: |
        sed -i "s|REPLACE_WITH_ECR_IMAGE|$IMAGE|g" aws/ecs-task-definition.json
        aws ecs register-task-definition \
          --cli-input-json file://aws/ecs-task-definition.json
        aws ecs update-service \
```

```
            --cluster ${{ secrets.ECS_CLUSTER }} \
            --service ${{ secrets.ECS_SERVICE }} \
            --force-new-deployment
```

The workflow has three jobs. The first builds, tests and builds the image. The second deploys to AKS. The third pushes to ECR and deploys to EKS and to ECS Fargate. The cloud jobs run only on the main branch and read every credential from GitHub secrets.

The real run output

The text below is the captured output of the verified run of this exact project on the build machine. The jar was built by Maven, started with the H2 profile on a test port, and exercised with real HTTP requests: two creates, one complete action, the list, the status filter, the stats endpoint and the frontend page. Nothing here is typed by hand or simulated.

output.txt

```
================================================================
TaskFlow - real build and run output
Captured on this machine from the actual jar, H2 local profile.
================================================================

--- Maven package result ---
-rw-rw---- 1 root nogroup 50268386 Nov 13 00:16 target/taskflow-1.0.0.jar
Tests (from target/surefire-reports):
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 9.676 s -- in
com.abel.taskflow.TaskflowApplicationTests
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.798 s -- in
com.abel.taskflow.controller.TaskControllerTest

--- Application startup log, key lines ---
2026-11-13T00:16:32.991Z  INFO 18310 --- [taskflow] [           main] com.abel.taskflow.TaskflowApplication    :
Starting TaskflowApplication v1.0.0 using Java 17.0.21 with PID 18310
(/home/hatch/workspace/taskflow-java/target/taskflow-1.0.0.jar started by root in
/home/hatch/workspace/taskflow-java)
2026-11-13T00:16:38.473Z  INFO 18310 --- [taskflow] [           main] com.zaxxer.hikari.HikariDataSource       :
HikariPool-1 - Starting...
2026-11-13T00:16:39.187Z  INFO 18310 --- [taskflow] [           main] com.zaxxer.hikari.pool.HikariPool        :
HikariPool-1 - Added connection conn0: url=jdbc:h2:mem:taskflow user=SA
2026-11-13T00:16:39.192Z  INFO 18310 --- [taskflow] [           main] com.zaxxer.hikari.HikariDataSource       :
HikariPool-1 - Start completed.
2026-11-13T00:16:39.776Z  INFO 18310 --- [taskflow] [           main] o.hibernate.jpa.internal.util.LogHelper  :
HHH000204: Processing PersistenceUnitInfo [name: default]
2026-11-13T00:16:43.246Z  INFO 18310 --- [taskflow] [           main] j.LocalContainerEntityManagerFactoryBean :
Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-11-13T00:16:44.807Z  INFO 18310 --- [taskflow] [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  :
Tomcat started on port 18081 (http) with context path '/'
2026-11-13T00:16:44.846Z  INFO 18310 --- [taskflow] [           main] com.abel.taskflow.TaskflowApplication    :
Started TaskflowApplication in 13.93 seconds (process running for 15.535)

--- POST /api/tasks : create a task ---
Request body:
{"title":"Design the database schema","description":"Tables for tasks and
users","status":"TODO","priority":"HIGH","dueDate":"2026-11-20"}
Response:
{"id":1,"title":"Design the database schema","description":"Tables for tasks and
users","status":"TODO","priority":"HIGH","dueDate":"2026-11-20","createdAt":"2026-11-13T00:16:46.58663132"}

--- POST /api/tasks : second task, in progress ---
{"id":2,"title":"Write the API documentation","description":"OpenAPI notes for the team","status":"IN_PROGRESS",
"priority":"MEDIUM","dueDate":"2026-11-25","createdAt":"2026-11-13T00:16:46.710311919"}

--- PATCH /api/tasks/2/complete ---
{"id":2,"title":"Write the API documentation","description":"OpenAPI notes for the
team","status":"DONE","priority":"MEDIUM","dueDate":"2026-11-25","createdAt":"2026-11-13T00:16:46.710312"}

--- GET /api/tasks ---
[{"id":1,"title":"Design the database schema","description":"Tables for tasks and users","status":"TODO","priori
ty":"HIGH","dueDate":"2026-11-20","createdAt":"2026-11-13T00:16:46.586631"},{"id":2,"title":"Write the API
documentation","description":"OpenAPI notes for the
team","status":"DONE","priority":"MEDIUM","dueDate":"2026-11-25","createdAt":"2026-11-13T00:16:46.710312"}]

--- GET /api/tasks?status=DONE ---
[{"id":2,"title":"Write the API documentation","description":"OpenAPI notes for the
team","status":"DONE","priority":"MEDIUM","dueDate":"2026-11-25","createdAt":"2026-11-13T00:16:46.710312"}]

--- GET /api/tasks/stats ---
{"total":2,"todo":1,"inProgress":0,"done":1}
```

```
--- GET / : the single-page frontend, HTTP status and title ---
HTTP 200, 6149 bytes
<title>TaskFlow - Team Task Manager</title>
```

Read the stats line closely. After two creates and one complete action the service reports total 2, to do 1, in progress 0 and done 1. The filter and the stats agree with the list, which is the behaviour the tests assert as well.

Deployment steps for Azure

Step 1. Create a resource group and an Azure Container Registry, then sign in to the registry with the Azure command line.

Step 2. Build the image in the cloud with the ACR build command, or build locally and push. Either way the registry now holds the taskflow image with a version tag.

Step 3. Create the AKS cluster with the registry attached, then download the cluster credentials for kubectl.

Step 4. Set the ACR image address in the deployment file, apply the five manifests from the k8s folder, and watch the rollout until both replicas report ready.

Step 5. For production, replace the in-cluster database with Azure Database for PostgreSQL and update the ConfigMap address. The full command list lives in the azure-deploy.md file in the project.

Deployment steps for AWS

Step 1. Create an ECR repository, sign Docker in to it, build the image, tag it with the full ECR address and push.

Step 2. Create an RDS PostgreSQL instance, store its password in Secrets Manager, and fill the task definition file with the real image address and database endpoint.

Step 3. Register the task definition, create the ECS cluster and the Fargate service with a desired count of two, and put an Application Load Balancer in front on port 8080.

Step 4. The Kubernetes route works too. Point kubectl at an EKS cluster and apply the same k8s folder. Only the image address changes. The full command list lives in the aws-deploy.md file in the project.

Limitations, stated honestly

The cloud deployments in this project are **configuration-ready**. The Dockerfile, the Compose file, the Kubernetes manifests, the ECS task definition, the deploy guides and the CI workflow are complete and consistent, and the application itself is built, tested and run-verified locally. No live Azure or AWS account was used while preparing the project, so no cloud resource was actually created and no cloud endpoint exists to visit.

The local verification used the H2 profile. The PostgreSQL path is standard Spring configuration and is exercised by Docker Compose and the cluster manifests, but it was not run against a live PostgreSQL server in this environment because no container runtime was available on the build machine.

The application has no login and no user accounts. It is a portfolio-grade team task manager, and authentication is the natural next step before any real team data goes into it.

The example Secret holds a demo password in plain base64. Base64 is an encoding, not encryption.

Trap: never commit a real password into a Secret file in a repository. Real deployments inject secrets from the cloud secret store at apply time.

Glossary

API. The set of HTTP addresses and rules that lets one program talk to another. TaskFlow exposes its tasks through an API.

Bean Validation. A Java standard for declaring rules on data, such as a required title, and having the framework enforce them before the data is stored.

CI and CD. Continuous integration and continuous delivery. Machines build, test and deploy the code on every change instead of a person doing it by hand.

ConfigMap. A Kubernetes object that carries plain settings, such as a database address, into a container as environment variables.

Container. A sealed package that holds an application and everything it needs to run, so it behaves the same on every machine.

CRUD. Create, read, update and delete. The four basic operations on stored data.

Deployment. A Kubernetes object that keeps a chosen number of application copies running and replaces them safely during an update.

Docker Compose. A tool that runs several containers together from one file, here the application and its database.

Entity. A Java class that maps to a database table. Each object of the class is one row.

Fargate. An AWS way of running containers without managing servers. AWS runs the container and bills for its size and time.

Ingress. A Kubernetes object that routes outside web traffic to services inside the cluster, by host name and path.

JAR. Java Archive. The single file that a Java application is packaged into and executed from.

JPA. Jakarta Persistence API. The Java standard for saving objects into a relational database.

JSON. A text format for structured data. The API and the page exchange tasks as JSON.

Kubernetes. A system that runs containers across a cluster of machines, restarts them when they fail and updates them without downtime.

Maven. The Java build tool that compiles the code, runs the tests and produces the jar, following the pom.xml file.

Multi-stage build. A Dockerfile pattern with a build stage and a slim runtime stage, so the shipped image stays small.

Profile. A named Spring configuration set. TaskFlow has a default local profile and a postgres profile.

Repository. In Spring Data, an interface that gives the code database operations for an entity without hand-written SQL. In Git, the home of the project history.

REST. A style for APIs where each address names a thing and the HTTP verb names the action on it.

Secret. A Kubernetes object that carries sensitive values, such as a database password, into a container.

Service. A Kubernetes object that gives a set of pods one stable network address and balances traffic across them.

ACR, AKS, ECR, ECS and EKS. The Azure and AWS homes of this project. ACR and ECR store images. AKS and EKS run Kubernetes. ECS runs containers as a service.

End of the explainer. The project, its tests, its captured run output and its deployment files all live in the taskflow-java project folder, ready to build.